

Programación Web

Seminario: Contenedores

Actualización: 17/4/2026



Contenido

- 
- Concepto de contenedor
 - Contenedores vs máquinas virtuales
 - Docker/Podman
 - Uso de contenedores en Programación Web
 - Mandatos de podman

Contenedores físicos



<https://blog.elinsignia.com/2016/11/22/contenedores-que-son-como-son-y-para-que-sirven/>

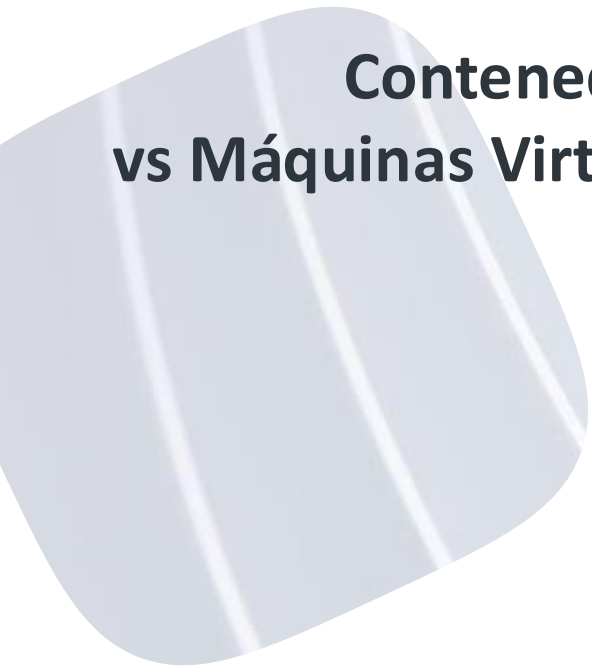
- Un **contenedor** es un **recipiente de carga** para el **transporte** marítimo o fluvial, transporte terrestre y transporte multimodal. Se trata de unidades estancas que protegen las mercancías de la climatología.
- Permite que una gran cantidad de **carga** sea **agrupada en una sola unidad**
- Las **dimensiones** del contenedor se encuentran **normalizadas** para facilitar su manipulación.

Definición



Unidad estándar de software que empaqueta **código** y *todas* sus **dependencias** de forma que la aplicación pueda ejecutarse rápida y fiablemente desde un entorno computacional a otro.

Una imagen de contenedor es un **paquete de software** ejecutable ligero y autónomo que incluye todo lo que necesita para ejecutar la aplicación (salvo el kernel del S.O.)



Contenedores vs Máquinas Virtuales

- En la virtualización con hipervisores, una o más máquinas se ejecutan virtualmente sobre el hardware a través de una capa de intermediación
- Los **contenedores se ejecutan en espacio de usuario**, sobre el kernel del S.O.
- Se pueden ejecutar **muchos** espacios de usuario aislados en un mismo equipo



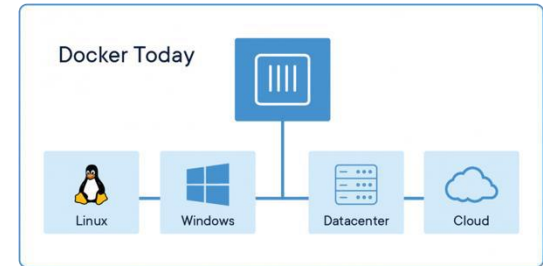
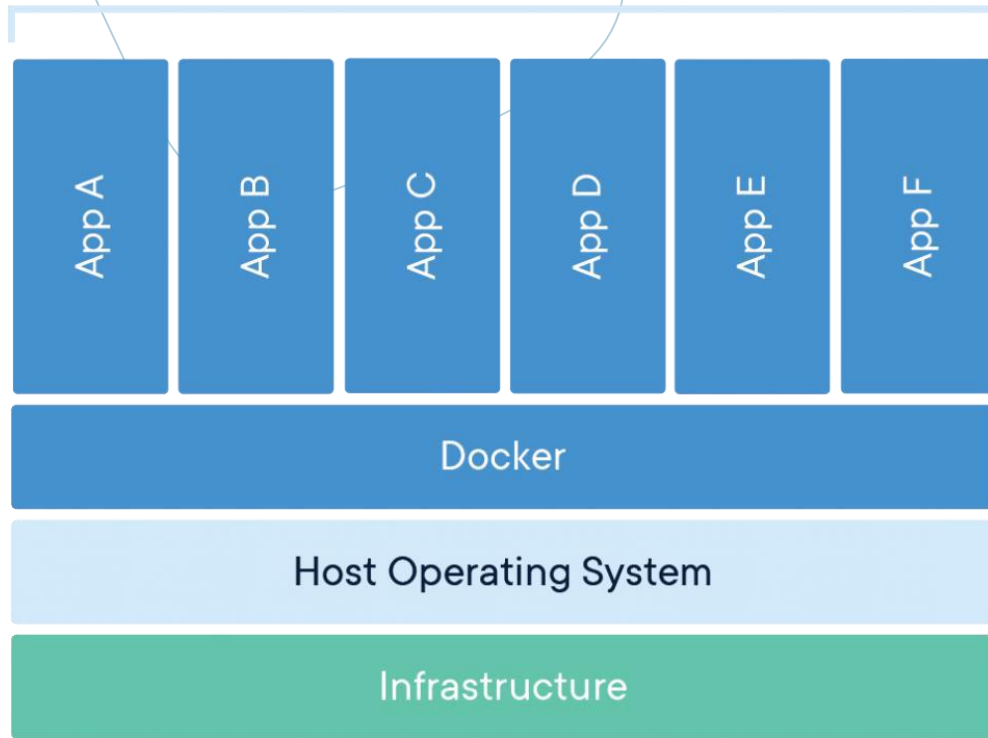
- Los contenedores son menos flexibles, pues sólo pueden ejecutar el mismo S.O. o similar al instalado en la máquina anfitriona
- Idea ya explorada en los años 1960

Docker



- Motor que facilita el despliegue de aplicaciones en contenedores
- Añade un motor de despliegue de aplicaciones sobre un entorno de ejecución virtualizada de contenedores
- Diseñado para poder desplegar aplicaciones de forma eficiente desde el entorno de desarrollo (¿portátil?) a producción

Containerized Applications

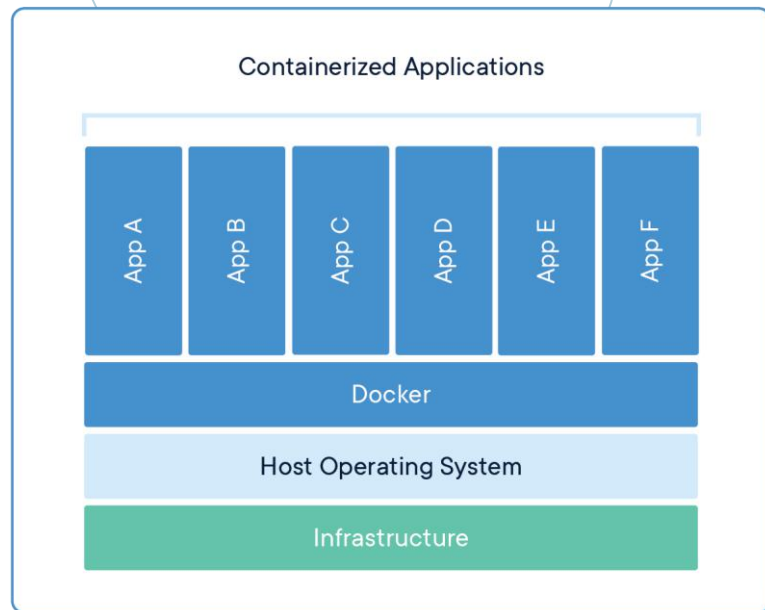


Contenedores

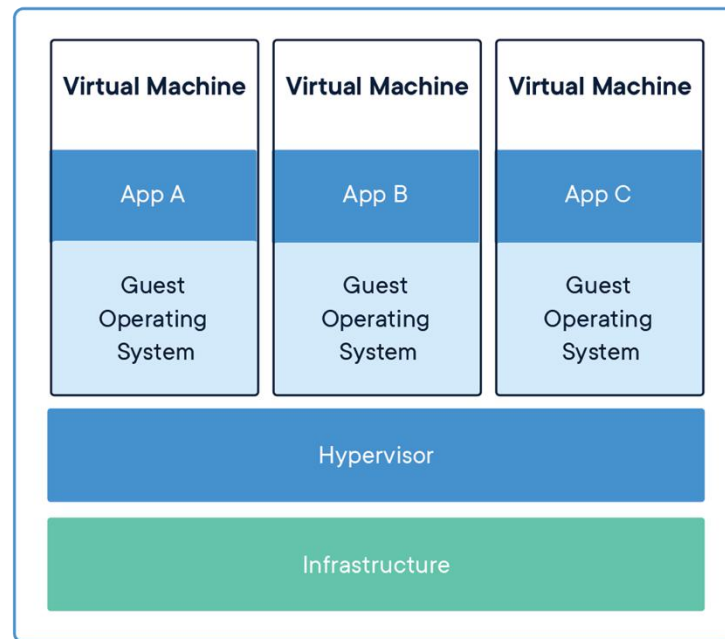


- Ubicación para el despliegue de aplicaciones y servicios
- Se “lanzan” y pueden tener uno o más procesos en ejecución
- Las imágenes → construyen contenedores
- Docker → ejecuta contenedores
- Son contenedores a imagen de los de mercancías, pero de software

Contenedores frente a máquinas virtuales



Abstracción a nivel de capa de aplicación.
Múltiples contenedores en la misma máquina
compartiendo S.O.



Abstracciones de hardware físico.
Múltiples servidores virtuales sobre uno físico
Cada MV tiene su copia del S.O.

Propiedades



- Separación lógica de tareas:
 - Desarrolladores: aplicaciones corriendo en contenedores
 - Administradores: gestión de contenedores
 - El entorno de desarrollo y producción es el **mismo**
- Ciclo de desarrollo rápido y eficiente
- Arquitectura orientada a servicios (SOA)

DevOps/ Continuous Integration

Usos habituales de contenedores

- Desarrollo local y eficiente de flujos de trabajo (Dev-Ops)
- Entornos de desarrollo ligeros y aislados
- Ejecución de servicios independientes y aislados
- Entornos aislados para prueba y validación
- Servicios PaaS y **SaaS**

Contenedores en Google

“Desde Gmail a YouTube, pasando por la Búsqueda, en Google todo se hace en contenedores. La creación de contenedores permite a nuestros equipos de desarrollo moverse con rapidez, desplegar software con eficacia y funcionar a una escala sin precedentes. Cada semana iniciamos más de varios miles de millones de contenedores.”

[→ Contenedores en Google](#)

Everything at Google runs in a container

The Managed Container Stack at Google

Managed Base OS

Node Container Manager

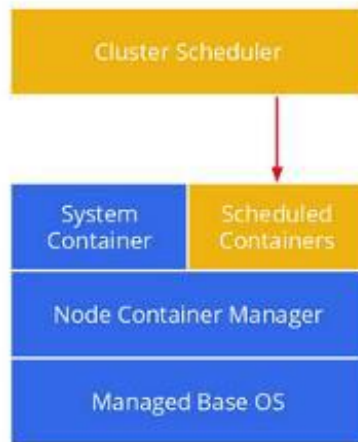
- Common services: log rotation, watchdog restarting

Containers:

- System container for shared daemons. Statically defined.
- Dynamically scheduled containers

Cluster Scheduler

- Schedules work (tasks) onto nodes
- Work specified based on intents
- Surfaces data about running tasks, restarts, etc.



Componentes de Docker



Servidor y clientes



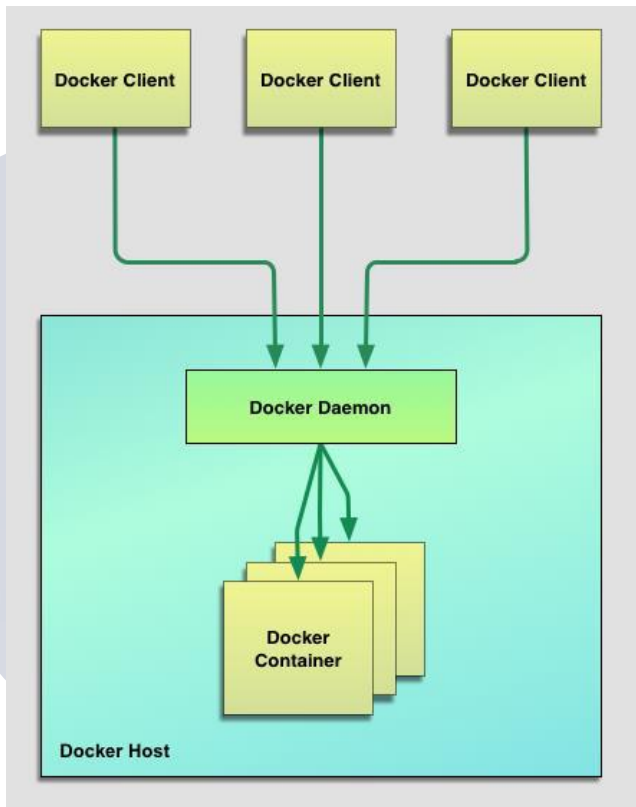
Imágenes



Registros



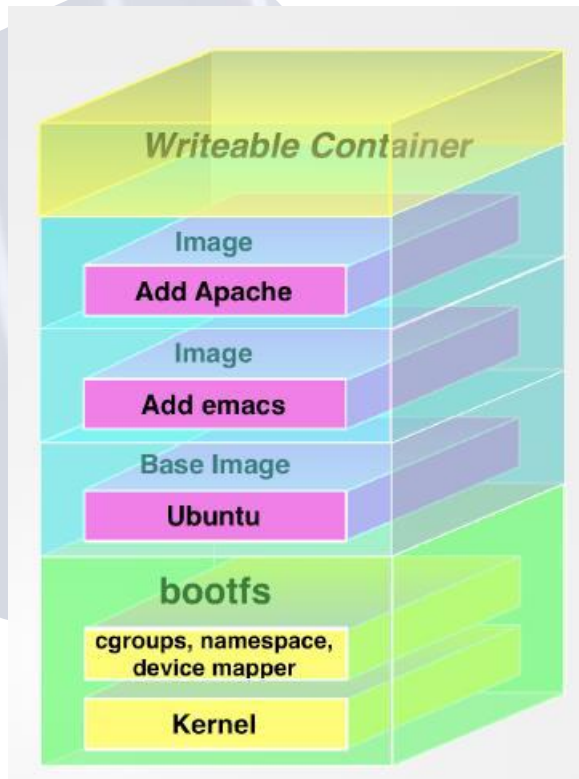
Contenedores



Cliente-servidor

- Servidor: demonio
- Cliente: programa (CLI)
docker

Imágenes

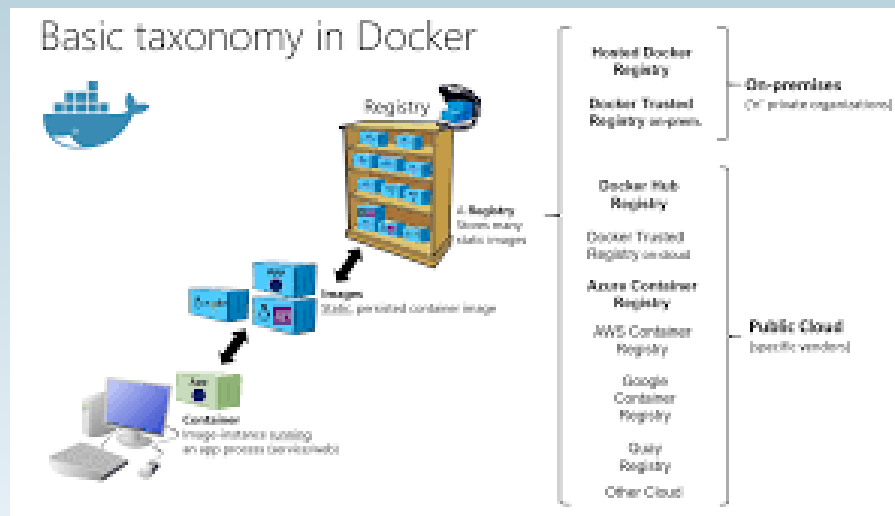


- “Código fuente” para los contenedores.
- Se construyen mediante la unión de sistemas de archivos
- Se apilan capas correspondientes a distintos sistemas de archivos

-> How containers work

Registros

- Repositorios de imágenes
- Hay públicos y privados
- Imágenes seguras
(**Docker Hardened Images**)
- Se pueden desplegar registros privados





Tareas habituales

- Descargar imágenes
- Ejecutar contenedores
- Administración de contenedores
- Creación de imágenes
- Orquestación

Podman



- Es un motor para el desarrollo, gestión y ejecución de contenedores **sin demonio**
- Soporta contenedores OCI (Open Container Initiative)
- Puede ejecutar contenedores sin privilegios de root
- <https://podman.io/>

-> Transitioning from Docker to Podman

Mandatos docker/podman

Descargar imágenes

```
podman pull nginx
```

docker = podman

Ejecutar contenedores

```
podman run -i -t centos /bin/bash
```

```
podman run -d nginx
```

```
podman run -d -p 8080:80 -name webserver nginx
```

Listado de contenedores en ejecución

```
podman ps
```

Mandatos docker (2)

Salvar un contenedor en una imagen nueva

```
podman commit nginx my_new_image
```

Listado de imágenes

```
podman images
```

Detener contenedor

```
podman stop <id>
```

Eliminar contenedor

```
podman kill <id>
```

Interacción con servidor web

Ejecutar un servidor web a contenedor en ejecución

```
podman run -d -p 20080:80 --rm --name my_webserver nginx
```

Acceder a contenedor en ejecución

```
podman exec -ti --name my_webserver /bin/bash
```

Instalar paquetes

```
apt-get update
```

```
apt-get install vim
```

Explorar el contenedor

```
cd /etc/nginx
```

```
more conf.d/default.conf
```

```
exit
```

Creación de imágenes

Creación del fichero Dockerfile

```
FROM ubuntu
MAINTAINER alumno
RUN apt-get update
RUN apt-get install -y apache2
RUN echo '<h1>Apache en un contenedor</h1>' >
/var/www/index.html
EXPOSE 80
ENTRYPOINT apache2 -D FOREGROUND
```

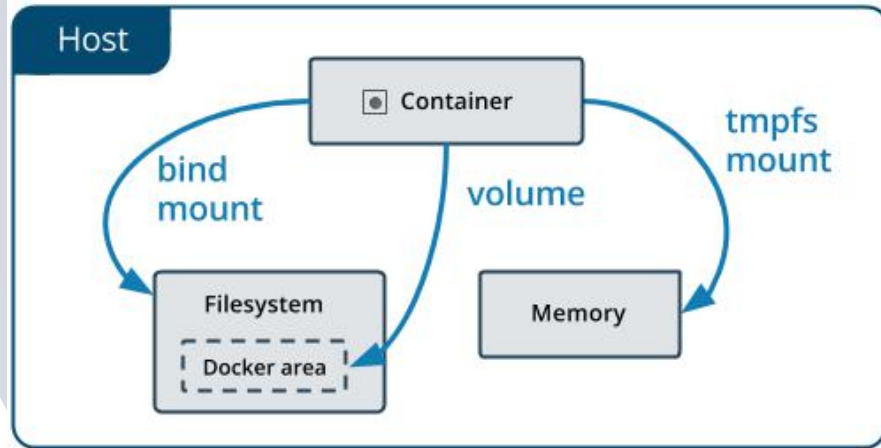
Construir contenedor

```
podman build -t alumno/apache .
```



-> How to create Docker images

Contenedores de datos



-> Manage data in Docker

-> Docker Data Containers

- Estructura de datos que almacena y organiza objetos virtuales.
- Permiten la migración de conjuntos de datos entre ordenadores.
- Persistencia de datos en contenedores de docker:
 - Volumes (contenedores)
 - Bind mounts (sistema de archivos)
 - tmpfs (memoria)

Despliegue de aplicación web

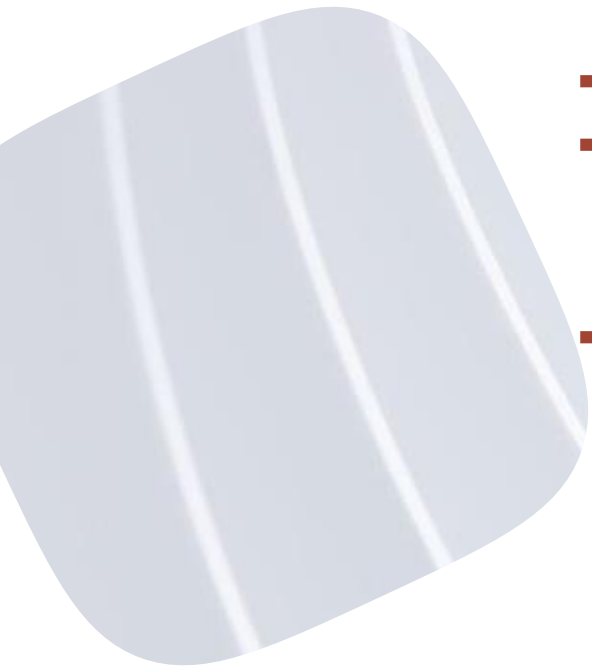
Poner contenidos en directorio local

```
mkdir /home/user/web  
vim index.html
```

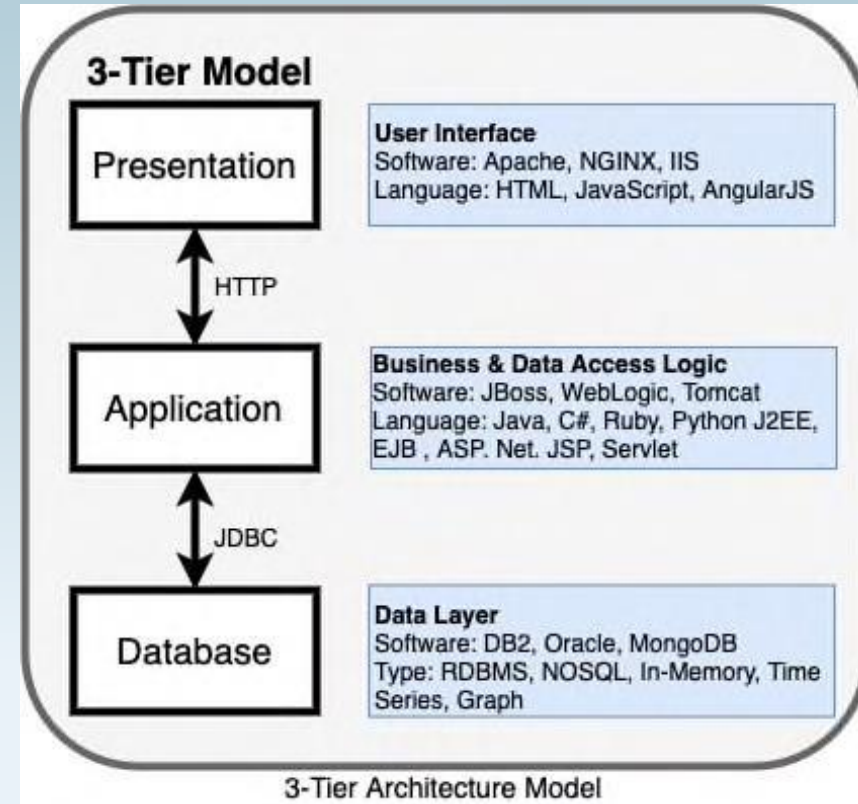
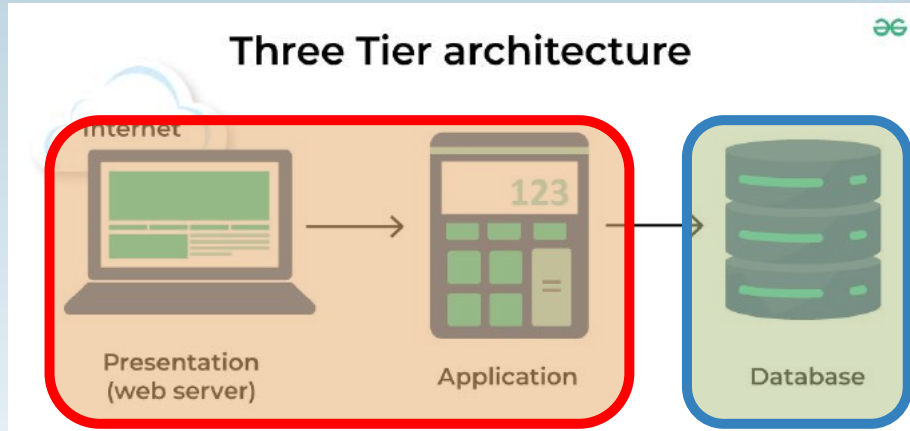
Ejecutar montando volumen externo

```
podman run -d --rm -v /home/user/web:/usr/share/nginx/html  
-p 20080:80 nginx
```

Docker compose

- 
- <https://docs.docker.com/compose/>
 - Es una herramienta para construir aplicaciones desplegadas sobre varios contenedores.
 - La especificación se escribe en un fichero YAML, que se “compila”.
 1. Definir el entorno con un Dockerfile
 2. Definir los servicios con docker-compose.yml
 3. Ejecutar docker compose up.

Arquitectura en 3 capas



Despliegue de aplicación web con base de datos

Crear “docker-compose.yml”

```
mkdir myapp
```

```
cd myapp
```

```
vim docker-compose.yml
```

Ejecutar montando volumen externo

```
podman-compose run
```

```
podman-compose up
```

```
podman-compose down
```

docker-compose.yml

services:

db:

image: mysql:5.7

environment:

MYSQL_ROOT_PASSWORD: "Klave\$"

MYSQL_DATABASE: test_db

MYSQL_USER: devuser

MYSQL_PASSWORD: "Secret#"

ports:

- "2006:3306"

web:

build:

context: .

dockerfile: ./dockerfile

container_name: php_web

depends_on:

- db

volumes:

- ./php/:/var/www/html/

ports:

- "20100:80"

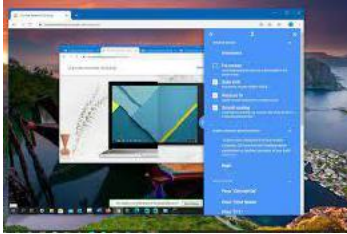
Lista de mandatos de docker

- <http://docs.docker.com/reference/commandline/cli>

Tutorial:

- https://github.com/DiCITS/MasterDataCom_BigData/blob/master/starting_docker.md

Ejecución de aplicaciones con docker



Tutoriales sobre contenedores



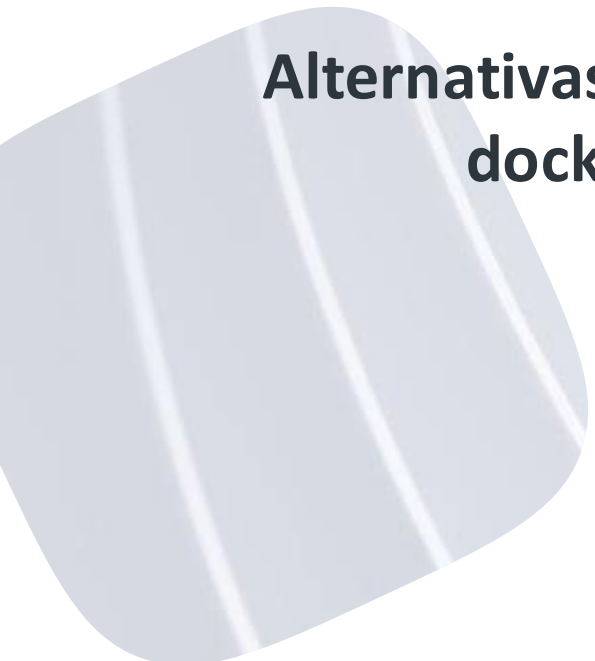
- <https://docker-curriculum.com/>
- <https://www.hostinger.es/tutoriales/como-crear-contenedor-docker>
- <https://www.youtube.com/watch?v=fqMOX6JJhGo>
- <https://www.simplilearn.com/tutorials/docker-tutorial>
- <https://aws.amazon.com/es/getting-started/hands-on/deploy-kubernetes-app-amazon-eks/>
- <https://iximiuz.com/en/posts/container-learning-path/>



OPEN CONTAINER INITIATIVE



- Estructura organizativa para el desarrollo de estándares para contenedores.
- Especificaciones actuales:
 - Runtime
 - Image



Alternativas a docker

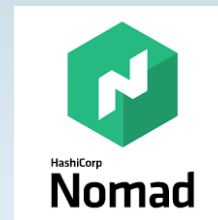
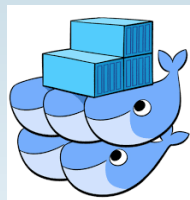
- CoreOS rkt
- Mesos Containerizer
- Podman
- LXC
- OpenVZ
- Linux Vserver
- Hyper-V Containers
- Unikernels
- ...

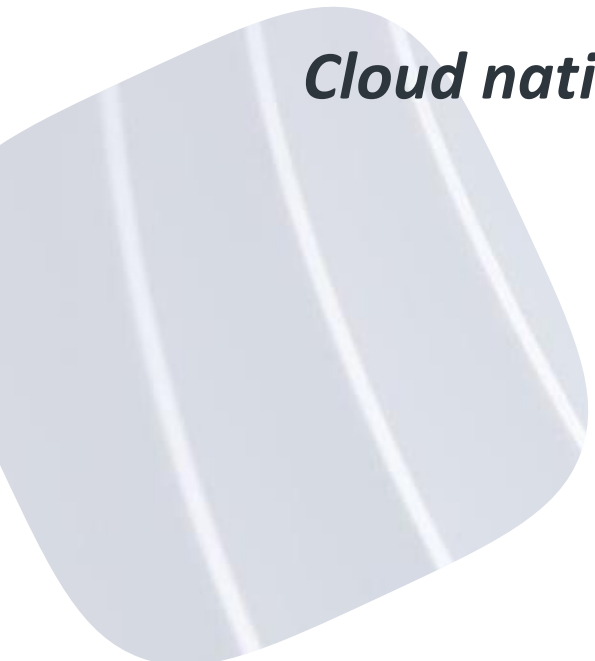
Orquestación de contenedores

- Interconexión de contenedores en una aplicación mayor para administrarla fácilmente
- Cooperación
- Balanceo de carga

Orquestadores

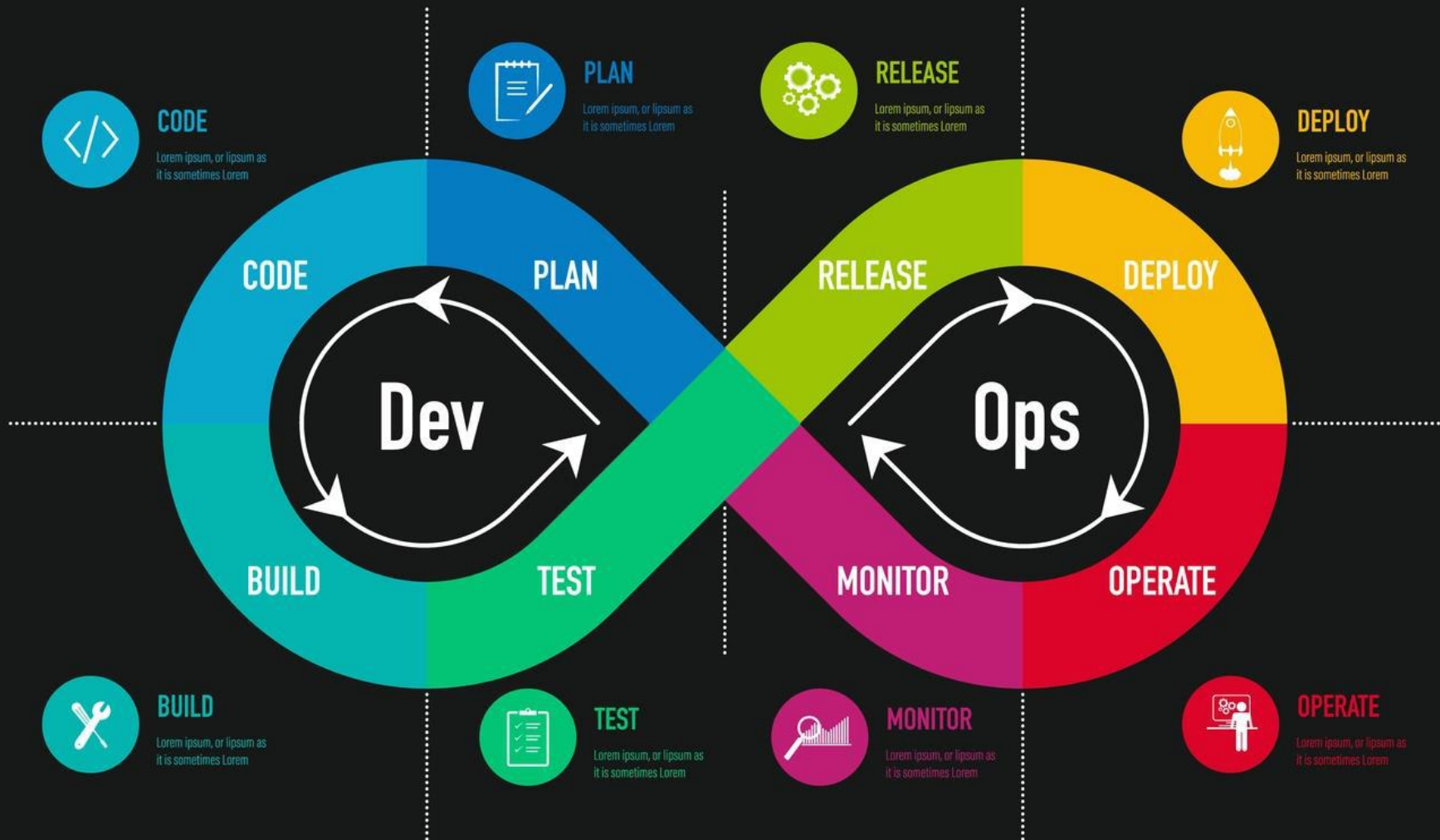
- Kubernetes
- Swarm
- Nomad



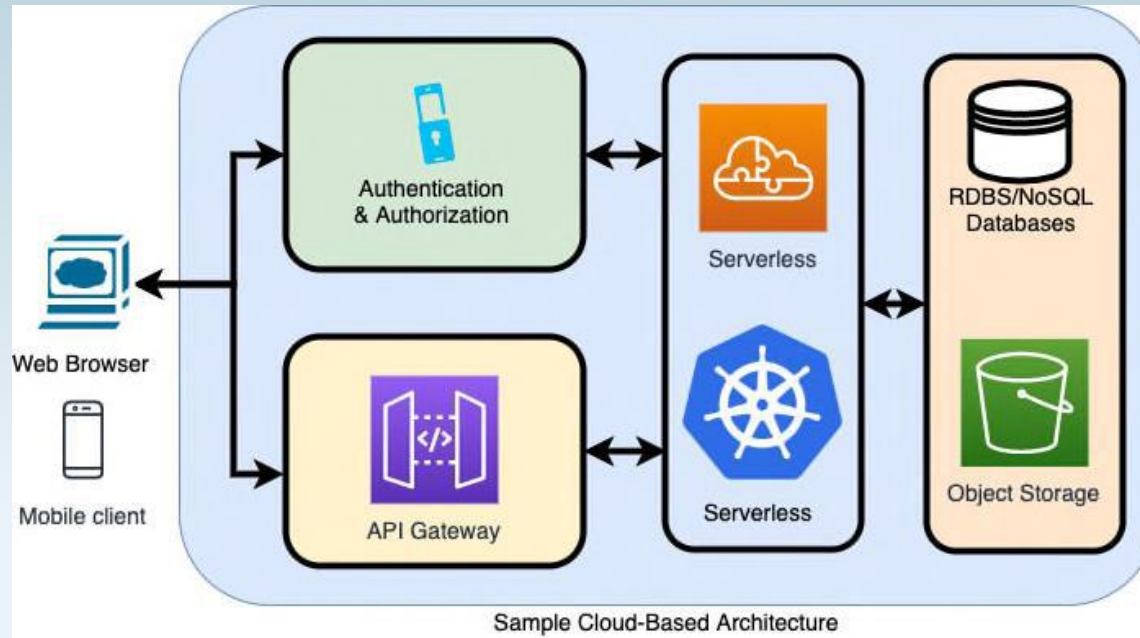


Cloud native

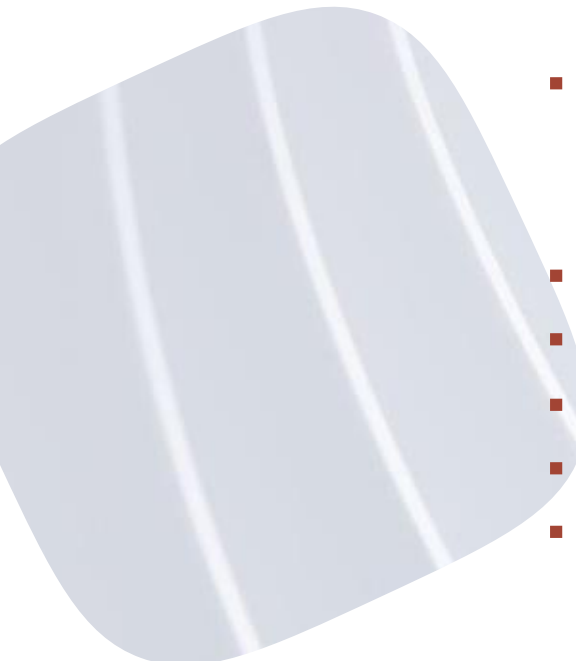
- Patrón de arquitectura software usado para el desarrollo y operación de servicios y aplicaciones en un entorno de computación en la nube.
- Elementos clave:
 - Contenedores
 - Orquestación de contenedores
 - Escalabilidad dinámica
 - Orientada a microservicios
 - DevOps y automatización



Arquitectura de aplicación basada en la nube



Bibliografía

- 
- Sean P. Kane, "Docker Up & Running: Shipping Reliable Containers in Production", O'Reilly, 2018
 - A. Mouat, "Using Docker", O'Reilly, 2016
 - I. Mell, "Docker in Practice", Manning, 2015
 - S. Goasquen, "Docker Cookbook", O'Reilly, 2015
 - J. Nickoloff, "Docker in Action", Manning, 2016
 - D. Walsh, "Podman in Action", Manning, 2023